

Stack & Linked List

ఇవి రెండూ చిన్న structures. అందుకే వీటి bugs కూడా చిన్నవి
– ఒక సంఖ్య, ఒక పంక్తి క్రమం, ఒక గుర్తు.

మొదటిది – కుండలీకరణాలని లెక్కపెట్టే పద్ధతి ఒకే రకం brackets మీద **0.00%** తప్పు, మరియు అది "సరైనవి" అన్న strings lo **81.9%** నిజానికి తప్పు. రెండోది – జాబితాని తిప్పేటప్పుడు రెండు పంక్తుల క్రమం మారిస్తే **10,000 nodes lo ఒక్కటే** మిగులుతుంది. మూడోది – monotonic stack **50,000x** వేగం, కానీ యాదృచ్ఛిక data మీద benchmark చేస్తే అది **5x** అని చెబుతుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే – మరియు §11 యొక్క విరుపుని నేను వెతకలేదు: నా స్వంత fuzz నా code మీద దాన్ని కనిపెట్టింది.

ఈ రెండూ ఒకే **DOC LO** ఎందుకు

Stack – "చివరిగా చూసినది ఏమిటి?" అనే ప్రశ్నకి జవాబు.

Linked list – "తర్వాతిది ఏమిటి?" అనే ప్రశ్నకి జవాబు.

రెండూ ఒక **pointer** ని పట్టుకుని నడవడం. మరియు రెండింటి bugs ఒకే ఆకారంలో ఉంటాయి: మీరు ఒక దాన్ని పట్టుకుని ఇంకొకదాన్ని వదిలేస్తారు – §7 lo అది మిగతా జాబితా, §4 lo అది bracket యొక్క రకం.

కాబట్టి ప్రశ్న ఒకటే: "ఈ క్షణంలో నేను ఏమి పట్టుకుని ఉన్నాను, మరియు దేన్ని వదిలేశాను?"

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Stack అంటే ఒక జ్ఞాపకం
2. అడగాల్సిన ప్రశ్నలు

Part 2 – మొదటి విరుపు: లెక్కపెట్టడం

3. Step – ఒక సంఖ్య చాలదా?
4. మొదటి విరుపు – 0.00%, ఆపై 81.9%
5. Step – stack ఏమి ఎక్కువ గుర్తుపెడుతుంది

Part 3 – రెండో విరుపు: pointer క్రమం

6. Step – జాబితాని తిప్పడం
7. రెండో విరుపు – 10,000 nodes, ఒక్కటే మిగులుతుంది
8. Step – recursion ఎందుకు కాదు

Part 4 – మూడో విరుపు: ఆకారం మరియు గుర్తు

9. Step – monotonic stack
10. మూడో విరుపు – 50,000×, కానీ benchmark 5× అంటుంది
11. **-1** అనే అబద్ధం – నా fuzz కనిపెట్టినది

Part 5 – నిరూపణ మరియు ప్రదర్శన

12. మొత్తం code
13. 24,000 cases · mutation testing

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Stack అంటే ఒక జ్ఞాపకం

Stack ఒక data structure కాదు – అది ఒక వాయిదా వేసే పద్ధతి.

"ఇది ఇప్పుడు తేల్చలేను. పక్కన పెడతాను. తర్వాత సమాచారం వచ్చాక తేల్చుతాను."

STACK ఎప్పుడు అవసరమో చెప్పే ఒక్క పరీక్ష

"ఈ మూలకం గురించి నిర్ణయం తీసుకోవడానికి, తర్వాత వచ్చేవాటిలో ఏదో ఒకటి కావాలా?"

అవును అంటే stack. ఎందుకంటే మీరు దాన్ని పక్కన పెట్టి ముందుకి వెళ్ళాలి, మరియు జవాబు వచ్చిన క్షణం వెనక్కి తిరిగి రావాలి.

(– ఇది సరైనదా? తర్వాత ఏ bracket వస్తుందో తెలియాలి. → **pending**

5 – దీనికి తర్వాత పెద్ద సంఖ్య ఏది? తర్వాతివి చూడాలి. → **pending**

మరియు "వెనక్కి తిరిగి రావడం" ఎప్పుడూ చివరిది ముందు జరుగుతుంది – అదే LIFO, అదే stack.

2. అడగాల్సిన ప్రశ్నలు

ప్రశ్న	ఎందుకు అడుగుతున్నాం
Brackets కి ఎన్ని రకాలు?	§4 – ఒకే రకం అయితే ఒక సంఖ్య చాలు, రెండైతే చాలదు
Input ఎంత పొడవు?	§8 – 10,000 దాటితే recursive రూపం RangeError
నకళ్ళు ఉంటాయా?	§10 – > ఆ >= ఆ అనేది జవాబుని 79.5% సార్లు మారుస్తుంది
విలువలు ఋణాత్మకం కావచ్చా?	§11 – -1 ని "జవాబు లేదు" అనే గుర్తుగా వాడితే అది ఒక అబద్ధం
జాబితాలో వలయం ఉండొచ్చా?	ఉంటే ఏ traversal అయినా ఆగదు – Floyd వాడాలి
తల (head) ని తీసేయాల్సి రావచ్చా?	రావచ్చు అంటే dummy తల – §12 lo ఒక mutation ఇదే

ప్రశ్న	ఎందుకు అడుగుతున్నాం
Input ఆకారం ఏమిటి – పెరుగుతూనా, తగ్గుతూనా?	§10 – ఇదే $1 \times$ మరియు $50,000 \times$ మధ్య తేడా

ఒక ప్రశ్న అడిగితే మరు దీన్ని నిజంగా నడిపారని తెలుస్తుంది

"input ఎలాంటి ఆకారంలో వస్తుంది – పెరుగుతూనా, తగ్గుతూనా, యాదృచ్ఛికంగానా?"

ఎందుకంటే §10 lo కొలిచినది ఇదే. అదే problem, అదే code, మూడు వేర్వేరు జవాబులు:

పెరుగుతూ → $1 \times$ (నేరుగా రెండు loops సరిపోతాయి) · **యాదృచ్ఛికం** → $5 \times$ · **తగ్గుతూ** → $50,000 \times$.

మీరు యాదృచ్ఛిక data మీద benchmark చేస్తే ప్రమాదాన్ని 10,000 రెట్లు తక్కువగా అంచనా వేస్తారు.

Part 2 – మొదటి విరుపు: లెక్కపెట్టడం

3. Step – ఒక సంఖ్య చాలదా?

"Brackets సరిగ్గా ఉన్నాయా?" – మొదటి ఆలోచన: లెక్కపెడదాం.

```
function valid(s) {  
  let n = 0;  
  for (const c of s) {  
    if (c === '(' || c === '[' || c === '{') n++;  
    else { n--; if (n < 0) return false; } // ముందే మూసేసాం  
  }  
  return n === 0;  
}
```

"()" ✓ · "()" ✓ · "()" ✓ · "()" ✗ · "()" ✗. నాలుగూ సరైనవి. Stack ఎందుకు?

4. మొదటి విరుపు – 0.00%, అపై 81.9%

కుండలీకరణాలు – "లెక్కపెడితే చాలు" అనే పద్ధతి

ఎన్ని రకాలు	50,000 యాదృచ్ఛిక strings	count తప్పు	%
1	50,000	0	0.00%
2	50,000	3,877	7.75%
3	50,000	4,603	9.21%

ఒకే రకం BRACKETS – సరిగ్గా ఉన్నా వైఫల్యాలు

() మాత్రమే ఉంటే లెక్కపెట్టడం నిజంగా సరిపోతుంది. అది "దాదాపు సరైనది" కాదు – అది నిరూపితంగా సరైనది.

కాబట్టి 33 lo నేను రాసిన నాలుగు ఉదాహరణలూ నెగ్గాయి, మరియు ఇంకో యాభై రాసినా నెగ్గేవి – అన్నీ () తోనే ఉన్నాయి కాబట్టి.

Deep 03, 05, 06, 08, 09 – ఇది ఆరోసారి. మరియు ఇక్కడ అత్యంత స్వచ్ఛమైన రూపంలో: మీ data ఒక చిన్న ఉపసమస్యని పరీక్షిస్తోంది, మరియు మీ code దానికి సరైనదే.

ఇప్పుడు హారం (denominator) మార్చుదాం – ఈ పద్ధతి "సరైనది" అన్న strings lo ఎన్ని నిజంగా తప్పు?

count "సరైనది" అన్న strings : 22,073
అందులో నిజానికి తప్పునవి : 18,073 (81.9%)

81.9% - ఈ FUNCTION "సరే" అన్న ప్రతి అయిదింటిలో నాలుగు తప్పు

"([)]" - లెక్క: ([]) → +1, +2, +1, 0. ముగింపు సున్నా, ఎప్పుడూ ఋణాత్మకం కాలేదు. "సరైనది."

కానీ [ని) తో మూశాం.

ఒక సంఖ్య ఎన్ని తెరిచి ఉన్నాయో గుర్తుపెడుతుంది. అది ఏవి తెరిచి ఉన్నాయో గుర్తుపెట్టదు.

"([{}])" కూడా అంతే - మూడు రకాలు, పరిపూర్ణ లెక్క పూర్తిగా తప్పు.

5. Step – stack ఏమి ఎక్కువ గుర్తుపెడుతుంది

```
const PAIR = { ')': '(', ']': '[', '}': '{' };  
function isValidBrackets(s) {  
  const st = [];  
  for (const c of s) {  
    if (c === '(' || c === '[' || c === '{') st.push(c);  
    else if (c in PAIR) { if (st.pop() !== PAIR[c]) return false; }  
  }  
  return st.length === 0;  
}
```

తేడా ఒక్కటే: `n++` కి బదులు `st.push(c)` - సంఖ్యకి బదులు అక్షరం.

STACK = COUNTER + జ్ఞాపకం

`st.length` సరిగ్గా పాత `n` నే. Stack ఆ సంఖ్యని కోల్పోలేదు - దానికి ఒక జాబితా కలిపింది.

అందుకే `st.pop() !== PAIR[c]` అనే ఒక్క పోలిక 81.9% ని సున్నా చేస్తుంది.

"ఒక సంఖ్య సరిపోతుందా, లేక జాబితా కావాలా?" - stack problems అన్నిటికీ ఇదే మొదటి ప్రశ్న.

Part 3 – రెండో విరుపు: pointer క్రమం

6. Step – జాబితాని తిప్పడం

మూడు pointers, ఒక loop:

```
function reverse(head) {  
  let prev = null, cur = head;  
  while (cur) {  
    cur.next = prev;    // వెనక్కి తిప్పు  
    prev = cur;         // ముందుకి జరుగు  
    cur = cur.next;  
  }  
  return prev;  
}
```

చదవడానికి సరైనదే – తిప్పు, జరుగు, జరుగు.

7. రెండో విరుపు – 10,000 nodes, ఒక్కటే మిగులుతుంది

Linked list ని తిప్పడం – రెండు పంక్తుల క్రమం మారిస్తే

n	సరైన code	తిప్పు code	మిగిలినవి
1	1	1	100.0%
2	2	1	50.0%
5	5	1	20.0%
100	100	1	1.0%
10000	10,000	1	0.0%

n = 5: సరైనది [5,4,3,2,1] · తప్పు [1]

N ఎంతైనా సరే – మిగిలేది సరిగ్గా ఒకటి

cur.next = prev అనే పంక్తి మిగతా జాబితాకి ఉన్న ఒకే ఒక్క దారిని తుడిచేస్తుంది.

ఆ తర్వాతి పంక్తి cur = cur.next ఇప్పుడు cur = prev చదువుతుంది – అంటే మనం ఇప్పుడే వదిలేసిన node. Loop మూడో సారి ఆగిపోతుంది.

మరియు n = 1 దగ్గర ఇది 100% సరైనది. ఒక node ఉన్న జాబితాతో పరీక్షిస్తే – నెగ్గుతుంది.

పరిష్కారం ఒక పంక్తి:

```
while (cur) {
  const nx = cur.next; // ← ముందు దాచు. దారిని తుడిచే *ముందు*.
  cur.next = prev;
  prev = cur;
  cur = nx;
}
```

ఇదే ఈ **DOC** యొక్క కేంద్ర వాక్యం

"ఈ క్షణంలో నేను ఏమి పట్టుకుని ఉన్నాను, మరియు దేన్ని వదిలేశాను?"

`cur.next` అనేది మిగతా 9,999 nodes కి ఉన్న ఏకైక దారి. దాన్ని మార్చే ముందు దాచకపోతే అవన్నీ పోతాయి – మరియు JavaScript ఏమీ చెప్పదు, garbage collector నిశ్శబ్దంగా వాటిని తీసేస్తుంది.

§4 lo వదిలేసినది bracket యొక్క రకం. ఇక్కడ వదిలేసినది మిగతా జాబితా. ఒకే తప్పు, రెండు రూపాలు.

8. Step – recursion ఎందుకు కాదు

తిప్పడాన్ని recursion తో రాయొచ్చు, మరియు అది అందంగా ఉంటుంది:

```
function reverseRec(head) {
  if (!head || !head.next) return head;
  const h = reverseRec(head.next);
  head.next.next = head;
  head.next = null;
  return h;
}
```

recursive రూపం · default stack

```
n = 1,000 | సరే · 1000 nodes
n = 5,000 | సరే · 5000 nodes
n = 10,000 | RangeError: Maximum call stack size exceeded
n = 20,000 | RangeError: Maximum call stack size exceeded
```

LINKED LIST RECURSION ఎప్పుడూ N లోతు – TREE లాగా LOG N కాదు

Deep 04 lo `fib` · **Deep 09** lo quicksort · **Deep 10** lo tree traversal – ప్రతిసారీ అదే `RangeError`, మరియు ప్రతిసారీ ~10,000 దగ్గరే.

తేడా: tree lo అది చెడ్డ ఆకారం వల్ల వస్తుంది. **Linked list lo ఆకారమే n లోతు** – తప్పించుకునే దారి లేదు.

కాబట్టి జాబితాల మీద recursion ఒక ఎంపిక కాదు. Iterative రూపమే ఏకైక రూపం.

Part 4 – మూడో విరుపు: ఆకారం మరియు గుర్తు

9. Step – monotonic stack

"ప్రతి మూలకానికీ, దాని కుడి వైపు వచ్చే మొదటి పెద్ద సంఖ్య ఏది?"

నేరుగా: ప్రతి i కి కుడి వైపు నడవడం – $O(n^2)$.

Stack తో: stack lo తగ్గని క్రమంలో indices పెట్టుకుని, ఒక పెద్ద సంఖ్య వచ్చినప్పుడు దానికంటే చిన్నవన్నీ బయటికి తీయడం – ప్రతి index ఒక్కసారే లోపలికి, ఒక్కసారే బయటికి, అంటే $O(n)$.

10. మూడో విరుపు – 50,000x, కానీ benchmark 5x అంటుంది

"తర్వాత వచ్చే పెద్ద సంఖ్య" · పోలికల సంఖ్య

input ఆకారం	n	రెండు loops	monotonic stack	రెట్లు
తగ్గుతూ	1000	4,99,500	999	500x
తగ్గుతూ	10000	4,99,95,000	9,999	5000x
తగ్గుతూ	100000	4,99,99,50,000	99,999	50000x
యౌదృచ్చికం	1000	6,931	1,988	3x
యౌదృచ్చికం	10000	88,067	19,981	4x
యౌదృచ్చికం	100000	10,36,994	1,99,970	5x
పెరుగుతూ	100000	99,999	99,999	1x

మూడు ఆకారాలు, మూడు వేర్వేరు కథలు

పెరుగుతూ → 1x. నేరుగా రెండు loops కూడా $O(n)$ – ఎందుకంటే తర్వాతి మూలకమే ఎప్పుడూ పెద్దది, లోపలి loop వెంటనే ఆగుతుంది.

తగ్గుతూ → 50,000x. ఇక్కడ ఏ మూలకానికీ జవాబు లేదు, కాబట్టి లోపలి loop ప్రతిసారీ చివరిదాకా నడుస్తుంది. 4,99,99,50,000 – అది సరిగ్గా $n(n-1)/2$.

యౌదృచ్చికం → 5x. మరియు ఇదే మీరు benchmark చేసేది.

రెట్లు 500 → 5,000 → 50,000: n పది రెట్లు పెరిగితే నిష్పత్తి పది రెట్లు. అదే $O(n^2)$ యొక్క సంతకం.

నేను ఈ పట్టికని మొదట తలకిందులుగా రాశాను

నా మొదటి కొలతలో *పెరుగుతున్న* array ని "worst case" అని రాశాను – "ప్రతి మూలకానికీ కుడి వైపు వెతకాలి కదా" అనుకున్నాను.

కొలత $1 \times$ చూపించింది. పెరుగుతున్న array నేరుగా loops కి అత్యుత్తమ సందర్భం.

ఏ ఆకారం చెడ్డదో నా ఊహ తప్పు. కొలత సరిచేసింది. అందుకే ఈ series lo ప్రతి సంఖ్య నడిపినదే – "ఇది స్పష్టంగా $O(n^2)$ " అని రాయడం సులభం, మరియు అది ఏ inputs మీద అనేది ఇంకో ప్రశ్న.

నకళ్ళు: `>` ఆ, `>=` ఆ?

[2,2,2] · "పెద్దది" [-1,-1,-1] · "పెద్దది లేదా సమానం" [2,2,-1]
నకళ్ళు ఉన్న 20,000 arrays lo రెండూ వేరే జవాబు ఇచ్చినవి: 15,905 (79.5%)

79.5%. ఇది bug కాదు – ఇది మీరు ఏ ప్రశ్న అడుగుతున్నారో అనేది. కానీ code చదివితే ఆ ప్రశ్న ఏమిటో తెలియదు, మరియు `>` ని `>=` గా మార్చడం ఒక్క అక్షరం.

11. -1 అనే అబద్ధం – నా fuzz కనిపెట్టినది

§13 కోసం fuzz రాసినప్పుడు ఒక నియమం పెట్టాను – "జవాబు `-1` అయితే, కుడి వైపు నిజంగా పెద్ద సంఖ్య ఉండకూడదు". నా స్వంత code మీద నడిపాను:

నియమ ఉల్లంఘనలు: 2
nextGreater[124] = -1, కానీ -1 ఉంది
nextGreater[125] = -1, కానీ -1 ఉంది

-1 అంటే "జవాబు లేదు" ఆ, లేక "జవాబు -1" ఆ?

పాఠ్యపుస్తక రూపం విలువని తిరిగి ఇస్తుంది, మరియు జవాబు లేకపోతే `-1` అంటుంది.

కానీ array lo `-1` ఒక నిజమైన విలువ కావచ్చు. అప్పుడు `-1` రెండు వేర్వేరు విషయాలు చెబుతుంది, మరియు పిలిచినవాడు వాటిని విడదీయలేడు.

`[-5, -1, -9] → విలువలు [-1,-1,-1]`. మొదటి `-1` నిజమైన జవాబు (`-1 > -5`). మిగతా రెండు "జవాబు లేదు". ఒకే సంఖ్య, రెండు అర్థాలు.

`-1` అస్పష్టమైన జవాబు ఎంత తరచుగా వస్తుంది?

విలువల పరిధి	50,000 arrays	అస్పష్టమైనవి	%
0 నుంచి 100 (LeetCode)	50,000	0	0.00%
-10 నుంచి 10	50,000	5,462	10.92%
-5 లక్షల నుంచి +5 లక్షలు	50,000	0	0.00%

మీ DATA దీన్ని రెండు వేర్వేరు కారణాలతో దాస్తుంది

0-100 → 0.00%, ఎందుకంటే LeetCode #496 lo $0 \leq \text{nums}[i]$ అని రాసి ఉంది. అక్కడ -1 సురక్షితం – నిర్వచనం ప్రకారం.

-5 లక్షల నుంచి +5 లక్షలు → 0.00%, ఎందుకంటే పది లక్షల విలువల్లో సరిగ్గా -1 తగలడం అరుదు. ఇది అదృష్టం, నిర్వచనం కాదు.

-10 నుంచి 10 → 10.92%. చిన్న పరిధి, ఋణాత్మక విలువలు – అంటే ఉష్ణోగ్రతలు, తేడాలు, offsets, స్కేరు మార్పులు. నిజమైన systems lo అత్యంత సాధారణ ఆకారం.

పరిష్కారం: విలువకి బదులు index ఇవ్వడం. Index ఎప్పుడూ ≥ 0 , కాబట్టి -1 కి ఎలాంటి ఊహ అక్కర్లేదు.

$[-5, -1, -9] \rightarrow [1, -1, -1]$. "index 1 lo ఉంది" – స్పష్టం.

Part 5 – నిరూపణ మరియు ప్రదర్శన

12. మొత్తం code

```
// తర్వాత వచ్చే పెద్ద సంఖ్య • monotonic stack
// *విలువ* కాదు, *index*. §11 చూడండి - index ఎప్పుడూ  $\geq 0$ , కాబట్టి -1 అస్పష్టం కాదు.
function nextGreater(a) {
  const out = new Array(a.length).fill(-1), stack = [];
  for (let i = 0; i < a.length; i++) {
    while (stack.length && a[i] > a[stack[stack.length - 1]]) out[stack.pop()] = i;
    stack.push(i);
  }
  return out;
}
```

```
// చివరి నుంచి n-వ node ని తీసేయడం • dummy తలతో
function removeNthFromEnd(head, n) {
  const dummy = { v: null, next: head }; // ← తలనే తీసేయాల్సి వస్తే ఇది కాపాడుతుంది
  let lead = dummy, trail = dummy;
  for (let i = 0; i < n; i++) { if (!lead.next) return head; lead = lead.next; }
  while (lead.next) { lead = lead.next; trail = trail.next; }
  trail.next = trail.next.next;
  return dummy.next;
}
```

ఆ `dummy` ఒక్క పంక్తి "తలని తీసేయడం" అనే ప్రత్యేక సందర్భాన్నే తుడిచేస్తుంది – ఇప్పుడు ప్రతి node కీ ఒక తల్లి ఉంది. తీసేస్తే **120/120** విఫలం.

13. 24,000 cases • mutation testing

పన్నెండు నియమాలు: **isValidBrackets** = పునరావృత తగ్గింపు సూచన, సమతుల్యమైన దాని చుట్టూ జత కలిపినా సమతుల్యమే, తిప్పితే తిరగబడాలి, ఒక్క node కూడా పోకూడదు, రెండుసార్లు తిప్పితే అసలుది, **hasCycle** = visited-Set సూచన, **cycleStart** = visited-Set సూచన, **harness** నిజంగా ఆ node నే కట్టిందా, **removeNthFromEnd** = array splice, **nextGreater** = నేరుగా రెండు loops, ప్రతి జవాబూ కుడి వైపు, పెద్దది, మరియు అతి దగ్గరిది, పోలికలు $\leq 2n$.

300 runs • 9,000 bracket strings • 7,500 జాబితాలు • 7,500 arrays
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

brackets	9,000
validBr	4,045

oneType	3,021	← §4 యొక్క "0.00%" దారి
lists	7,500	
cyclic	3,662	
acyclic	3,838	
nge	7,500	
ngeDesc	1,876	← §10 యొక్క worst case
ngeDup	1,856	
removes	7,332	
removeHead	859	← తలనే తీసేసిన సందర్భాలు

==== mutation testing ====

మార్పు లేని code → 0/300 విఫలం
 isValidBrackets lo *రకం* చూడకుండా లెక్కపెడితే (§4 యొక్క bug) → 63/120 విఫలం
 ఉదా: isValidBrackets("([{}])") = true, సూచన false
 isValidBrackets lo చివరి `st.length === 0` తనిఖీ తీసేస్తే → 106/120 విఫలం
 ఉదా: isValidBrackets("((()))()") = true, సూచన false
 reverseList lo `nx` దాచకుండా `cur.next` ని ముందే మార్చితే → 120/120 విఫలం
 ఉదా: reverseList [86] ≠ [24,42,47,62,67,96,70,36,3,46,64,62,10,36,86]
 reverseList lo `return prev` ని `return head` గా → 120/120 విఫలం
 hasCycle lo `fast & fast.next` ని `fast` గా → 119/120 విఫలం
 ఉదా: TypeError: Cannot read properties of null (reading 'next')
 cycleStart lo `a = head` ని `a = slow` గా → 120/120 విఫలం
 nextGreater lo `a[i] >` ని `a[i] >=` గా → 120/120 విఫలం
 nextGreater lo index కి బదులు *విలువ* ఇస్తే (§11 యొక్క bug) → 120/120 విఫలం
 nextGreater lo `while` ని `if` గా (ఒక్కటే pop) → 120/120 విఫలం
 removeNthFromEnd lo dummy తల తీసేస్తే → 120/120 విఫలం
 removeNthFromEnd lo `i < n` ని `i <= n` గా → 120/120 విఫలం

రెండు MUTATIONS 120/120 కి చేరలేదు – ఎందుకొ

"రకం చూడకుండా లెక్కపెడితే" – 63/120. ఆ mutation విఫలం కావాలంటే ఒక run lo **రెండు కంటే ఎక్కువ రకాల brackets** ఉన్న, మరియు **సరిగ్గా తప్పుగా జతపడిన** string రావాలి. నా corpus lo మూడోవంతు strings ఒకే రకం (oneType 3,021) – అవి ఈ mutation కి **సరైన జవాబే** ఇస్తాయి.

అంటే **52.5% అనే సంఖ్య §4 యొక్క కొలతనే మళ్ళీ చెబుతోంది**: ఒకే రకం data ఈ bug ని చూడనే చూడదు.

hasCycle lo fast మాత్రమే – 119/120. ఒక run lo అన్ని జాబితాలకీ వలయం ఉంటే **fast.next** ఎప్పుడూ **null** కాదు, మరియు crash రాదు. 300 runs lo అది ఒకసారి జరిగింది.

రెండూ దాచడం లేదు – రెండూ కొలిచి, కారణం చెప్పినవి.

14. Follow-ups – deque, LRU, skip list

"రెండు చివర్ల నుంచీ కావాలంటే?"

Deque (double-ended queue). JavaScript lo **Array.shift()** **O(n)** – ముందు నుంచి తీస్తే మిగతావాన్నీ జరగాలి. నిజమైన **O(1)** deque కి **వలయాకార buffer** లేదా **doubly linked list** కావాలి. **నేను**

`shift()` ఖరీదుని ఈ `doc lo` కొలవలేదు – LLD Deep 14 (Text Editor) lo array కాపీ ఖరీదు కొలిచాను.

"LRU cache కి ఇవి ఎలా కలుస్తాయి?"

Doubly linked list + hash map: map ఇస్తుంది $O(1)$ వెతుకులాట, list ఇస్తుంది $O(1)$ తొలగింపు మరియు చేర్పు. LLD Deep 02 lo దీన్ని పూర్తిగా కొలిచాను – **ఒక్క read 183 ms $\rightarrow$ 1 ms.**

"Linked list మీద binary search చేయొచ్చా?"

కాదు – మధ్యకి దూకలేము, నడవాల్సిందే. అందుకే **skip list** ఉంది: అనేక అంతస్తుల జాబితాలు, పైవి ఎక్కువ దూరం దూకుతాయి, సగటున $O(\log n)$. నేను **skip lists ని కొలవలేదు.**

"Floyd యొక్క రెండే దశ ఎందుకు పనిచేస్తుంది?"

తలనుంచి వలయం మొదలుకి దూరం `a`, మొదలునుంచి కలిసిన చోటికి `b`, వలయం పొడవు `c` అనుకుంటే – కలిసిన క్షణంలో $a = (k \cdot c) - b$. కాబట్టి **తల నుంచి మరియు కలిసిన చోటి నుంచి** ఒకే వేగంతో నడిస్తే సరిగ్గా మొదలు దగ్గర కలుస్తాయి. §13 lo ఒక నియమం దీన్ని ప్రతి run lo visited-Set తో సరిచూస్తుంది – **నిరూపణ నమ్మి ఊరుకోలేదు.**

"ఈ `doc lo` నేను ఏమి కొలవలేదు"

మూడు: **deque మరియు `shift()` ఖరీదు, skip lists, మరియు doubly linked lists** – ఇక్కడ అంతా singly.

15. ఏమి నేర్చుకున్నాం

1. "ఒక సంఖ్య సరిపోతుందా, లేక జాబితా కావాలా?" – stack problems అన్నిటికీ ఇదే మొదటి ప్రశ్న.
2. ఒకే రకం brackets మీద లెక్కపెట్టడం నిరూపితంగా సరైనది – **0.00%** తప్పు. రెండే రకం వచ్చిన క్షణం అది విరుగుతుంది.
3. ఆ function "సరే" అన్న strings lo **81.9%** తప్పు. `[]]` – పరిపూర్ణ లెక్క పూర్తిగా తప్పు.
4. `cur.next` మిగతా జాబితాకి ఏకైక దారి. దాన్ని మార్చే ముందు దాచకపోతే **10,000 nodes lo ఒక్కటే** మిగులుతుంది – మరియు $n = 1$ దగ్గర అది 100% సరైనది.
5. జాబితాల మీద recursion ఒక ఎంపిక కాదు. లోతు ఎప్పుడూ n , మరియు `RangeError` 10,000 దగ్గర.
6. Input ఆకారం $1 \times$ మరియు $50,000 \times$ మధ్య తేడా. పెరుగుతూ $\rightarrow 1 \times$, యాదృచ్ఛికం $\rightarrow 5 \times$, తగ్గుతూ $\rightarrow 50,000 \times$.
7. యాదృచ్ఛిక data మీద benchmark చేస్తే ప్రమాదం 10,000 రెట్లు తక్కువగా కనిపిస్తుంది.
8. ఏ ఆకారం చెడ్డదో నా ఊహ తప్పు. నేను పెరుగుతున్నది worst అనుకున్నాను; అది best. **కొలత సరిచేసింది.**
9. `-1` ని "జవాబు లేదు" అనే గుర్తుగా వాడటం ఒక ప్రకటించని ఊహ. LeetCode lo ఆ ఊహ రాసి ఉంది; మీ code lo లేదు. **Index ఇవ్వండి** – అది ఎప్పుడూ ≥ 0 .

10. > ఆ >= ఆ అనేది 79.5% సార్లు జవాబుని మారుస్తుంది – అది bug కాదు, అది మీరు ఏ ప్రశ్న అడుగుతున్నారు అనేది.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

Stack మరియు linked list code lo ప్రతి పంక్తి ఒక pointer ని పట్టుకుంటుంది మరియు ఇంకొకదాన్ని వదిలేస్తుంది. Bug ఎప్పుడూ వదిలేసినదాంట్లోనో.

§4 lo వదిలేసినది bracket యొక్క రకం · §7 lo మిగతా జాబితా · §11 lo **-1** అంటే ఏమిటో.

మూడింటిలోనూ code నడిచింది, crash కాలేదు, మరియు ఒక జవాబు ఇచ్చింది. కాబట్టి ప్రతి పంక్తి తర్వాత అడగండి: **"ఇప్పుడు నేను దేన్ని వదిలేశాను, మరియు అది నాకు మళ్ళీ కావాలా?"**